

Built as a second sheet for questions that twist/combine lab concepts. Model -> modify inputs/parameters.

Source topics include feature extraction, convolution, edge detection, segmentation, transformations, wavelets, Hough and SIFT.

1. MANUAL GRADIENTS - SOBEL / PREWITT / SCHARR

SOBEL

```
gray=cv2.imread("image.jpg",0)
gx=cv2.Sobel(gray,cv2.CV_64F,1,0,ksize=3)
gy=cv2.Sobel(gray,cv2.CV_64F,0,1,ksize=3)
mag=cv2.magnitude(gx.astype(np.float32),
gy.astype(np.float32))
mag=cv2.convertScaleAbs(mag)
```

PREWITT (manual kernels)

```
kx=np.array([[ -1,0,1],[ -1,0,1],[ -1,0,1]],np.float32)
ky=np.array([[ -1,-1,-1],[0,0,0],[1,1,1]],np.float32)
gx=cv2.filter2D(gray,cv2.CV_32F,kx)
gy=cv2.filter2D(gray,cv2.CV_32F,ky)
mag=cv2.magnitude(gx,gy)
mag=cv2.convertScaleAbs(mag)
```

SCHARR

```
gx=cv2.Scharr(gray,cv2.CV_64F,1,0)
gy=cv2.Scharr(gray,cv2.CV_64F,0,1)
mag=cv2.magnitude(gx.astype(np.float32),
gy.astype(np.float32))
mag=cv2.convertScaleAbs(mag)
```

2. LAPLACIAN / LoG / EDGE COMPARISON

LAPLACIAN

```
lap=cv2.Laplacian(gray,cv2.CV_64F)
lap=cv2.convertScaleAbs(lap)
```

second derivative

LoG = Gaussian then Laplacian

```
blur=cv2.GaussianBlur(gray,(5,5),0)
log=cv2.Laplacian(blur,cv2.CV_64F)
log=cv2.convertScaleAbs(log)
```

EDGE PIPELINE

```
blur=cv2.GaussianBlur(gray,(5,5),0)
```

```
sobelx=cv2.Sobel(blur,cv2.CV_64F,1,0,3)
sobely=cv2.Sobel(blur,cv2.CV_64F,0,1,3)
mag=cv2.magnitude(sobelx.astype(np.float32),
sobely.astype(np.float32))
edges=cv2.Canny(blur,50,150)
```

3. CONVOLUTION - MANUAL / KERNEL / OUTPUT SIZE

MANUAL 2D CONVOLUTION

```
kernel=np.array([[1,0,-1],[1,0,-1],[1,0,-1]],float)
kh,kw=kernel.shape
out=np.zeros((gray.shape[0]-kh+1,
gray.shape[1]-kw+1))
for i in range(out.shape[0]):
for j in range(out.shape[1]):
out[i,j]=np.sum(gray[i:i+kh,j:j+kw]*kernel)
out=np.uint8(np.clip(out,0,255))
```

OPENCV FILTER

```
kernel=np.ones((3,3),np.float32)/9
filtered=cv2.filter2D(gray,-1,kernel)
# Gaussian:
filtered=cv2.GaussianBlur(gray,(5,5),0)
```

OUTPUT SIZE

valid convolution:

```
# out = floor((N-K+2P)/S)+1
```

```
N=32; K=3; P=0; S=1
```

```
out_size=(N-K+2*P)//S+1
```

```
print(out_size)
```

```
# know: padding P, stride S, kernel K
```

4. HOG - ACTUAL FEATURE VECTOR TEMPLATE

```
from skimage.feature import hog
from skimage import color
gray=color.rgb2gray(img)
features,visual=hog(
gray,orientations=9,
```

```

pixels_per_cell=(8,8),
cells_per_block=(2,2),
visualize=True)
print(features.shape)

```

HOG WORKFLOW

```

# image -> gradient magnitude/direction
# -> cells -> orientation histograms
# -> blocks -> normalized descriptor
#

```

Change:

```

# orientations=9
# pixels_per_cell=(8,8)
# cells_per_block=(2,2)

```

SHOW HOG

```

plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
plt.imshow(gray,cmap="gray"); plt.axis("off")
plt.subplot(1,2,2)
plt.imshow(visual,cmap="gray"); plt.axis("off")
plt.show()

```

5. LBP - TEXTURE FEATURE TEMPLATE

```

from skimage.feature import local_binary_pattern
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
P=8; R=1
lbp=local_binary_pattern(gray,P,R,method="uniform")
# histogram of LBP
hist,_=np.histogram(
    lbp.ravel(),bins=np.arange(0,P+3),
    range=(0,P+2))
hist=hist.astype(float)
hist/=hist.sum()

```

VISUALIZE

```

plt.imshow(lbp,cmap="gray")
plt.axis("off"); plt.show()
# P = number of neighbors
# R = radius
# uniform = common compact pattern

```

TOUGH TWIST:

Compare texture in two ROIs

```

a=lbp[y1:y2,x1:x2]
b=lbp[y3:y4,x3:x4]
ha,_=np.histogram(a.ravel(),bins=np.arange(0,P+3))
hb,_=np.histogram(b.ravel(),bins=np.arange(0,P+3))
print(np.linalg.norm(ha-hb))

```

6. COLOR HISTOGRAM / HED / HIG-STYLE FEATURES

COLOR HISTOGRAM

```

for c in range(3):
    hist=cv2.calcHist([img],[c],None,[256],[0,256])
    plt.plot(hist)
    plt.show()
# BGR channels; convert RGB if needed

```

GRADIENT DIRECTION HISTOGRAM

```

gx=cv2.Sobel(gray,cv2.CV_32F,1,0,3)
gy=cv2.Sobel(gray,cv2.CV_32F,0,1,3)
mag,ang=cv2.cartToPolar(gx,gy,angleInDegrees=True)
hist,_=np.histogram(ang,bins=9,range=(0,360),weights=mag)

```

HIG IDEA

```

# HIG = intensity gradients
# gradient magnitude weights the direction bins
#
# HOG/HIG twist:
# compute gx, gy -> magnitude + angle
# -> histogram -> feature vector

```

7. REGION GROWING - HARD SEGMENTATION QUESTION

SIMPLE REGION GROWING

```

seed=(y0,x0)
tol=10
visited=np.zeros(gray.shape,bool)
region=np.zeros(gray.shape,np.uint8)
queue=[seed]
seed_val=int(gray[seed])
while queue:
    y,x=queue.pop(0)

```

```

if visited[y,x]: continue
visited[y,x]=True
if abs(int(gray[y,x])-seed_val)<=tol:
    region[y,x]=255
for dy,dx in [(-1,0),(1,0),(0,-1),(0,1)]:
    ny,nx=y+dy,x+dx
    if 0<=ny<gray.shape[0] and 0<=nx<gray.shape[1]:
        if not visited[ny,nx]: queue.append((ny,nx))

```

CHANGE CONNECTIVITY

4-neighbor:

```
[(-1,0),(1,0),(0,-1),(0,1)]
```

8-neighbor:

```

[(dy,dx) for dy in (-1,0,1)
 for dx in (-1,0,1) if (dy,dx)!=(0,0)]
# stricter similarity -> smaller tol

```

EXAM LOGIC

```

# seed -> inspect neighbors
# -> if similar, add
# -> continue from new pixels
# -> stop when neighbors fail criterion
# seed can be:
seed=(100,100)
# similarity can use intensity/color distance

```

8. WATERSHED - COMPLETE TOUGH TEMPLATE

```

gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
_,b=cv2.threshold(gray,0,255,
cv2.THRESH_BINARY_INV+cv2.THRESH_OTSU)
kernel=np.ones((3,3),np.uint8)
opening=cv2.morphologyEx(b,cv2.MORPH_OPEN,kernel,2)
sure_bg=cv2.dilate(opening,kernel,3)

dist=cv2.distanceTransform(opening,cv2.DIST_L2,5)
_,fg=cv2.threshold(dist,0.5*dist.max(),255,0)
fg=np.uint8(fg)
unknown=cv2.subtract(sure_bg,fg)
n,markers=cv2.connectedComponents(fg)
markers+=1
markers[unknown==255]=0

```

```

markers=cv2.watershed(img,markers)
img[markers==-1]=(255,0,0)
# KEY:
# touching objects -> watershed
# distance transform -> sure foreground
# markers -> separation

```

9. ACTIVE CONTOUR / SNAKE - MODEL

skimage model

```

from skimage.segmentation import active_contour
from skimage.filters import gaussian
import numpy as np
g=gaussian(gray,3)
s=np.linspace(0,2*np.pi,100)
r=150+80*np.sin(s)
c=200+80*np.cos(s)
init=np.array([r,c]).T
snake=active_contour(
g,init,alpha=0.01,beta=10,
gamma=0.001)

```

VISUALIZE

```

plt.imshow(gray,cmap="gray")
plt.plot(init[:,1],init[:,0],"--r")
plt.plot(snake[:,1],snake[:,0],"-b")
plt.axis("off"); plt.show()
# idea: snake moves toward object boundary
# energy = internal + external forces

```

EXAM USE:

```

# if asked "segment an object boundary"
# and specifically mentions snakes/active contour:
# initialize curve around object
# run active_contour
# plot final contour

```

10. DWT / MULTI-RESOLUTION / ANOMALY

```

import pywt
LL,(LH,HL,HH)=pywt.dwt2(gray,"haar")
plt.figure(figsize=(8,6))

```

```
plt.subplot(2,2,1); plt.imshow(LL,cmap="gray")
plt.subplot(2,2,2); plt.imshow(LH,cmap="gray")
plt.subplot(2,2,3); plt.imshow(HL,cmap="gray")
plt.subplot(2,2,4); plt.imshow(HH,cmap="gray")
plt.show()
```

ANOMALY FROM HIGH-FREQUENCY DETAIL

```
detail=np.abs(HH)
T=detail.mean()+2*detail.std()
anom=(detail>T).astype(np.uint8)*255
# increase 2 -> stricter
# HH = diagonal detail
```

RECONSTRUCTION

```
rec=pywt.idwt2((LL,(LH,HL,HH)),"haar")
rec=np.uint8(np.clip(rec,0,255))
# DWT:
# image -> LL + LH + HL + HH
# inverse -> reconstruct image
```

11. CWT / WPT - KNOW THE CODE PATTERN

CWT (1D signal)

```
coef,freq=pywt.cwt(
    signal,
    scales=np.arange(1,64),
    wavelet="morl")
# coef = time/scale representation
# plot:
plt.imshow(abs(coef),aspect="auto",
    cmap="gray")
```

WAVELET PACKET

```
wp=pywt.WaveletPacket(
    data=gray, wavelet="haar",
    mode="symmetric",maxlevel=2)
for node in wp.get_level(2,"freq"):
    print(node.path,node.data.shape)
# WPT recursively decomposes sub-bands
```

WHEN TO USE

DWT -> standard multi-resolution decomposition
CWT -> continuous scale/translation analysis

WPT -> deeper recursive frequency sub-bands
If exact lab API is not given,
remember the conceptual decomposition.

12. HOMOGRAPHY FROM POINTS - HARD TWIST

```
src=np.float32([
    [100,100],[500,100],
    [500,400],[100,400]])
dst=np.float32([
    [130,80],[520,120],
    [480,430],[90,390]])
H,status=cv2.findHomography(
    src,dst,cv2.RANSAC,5.0)
warped=cv2.warpPerspective(
    img,H,(img.shape[1],img.shape[0]))
```

MAP A POINT

```
p=np.float32([[250,200]])
mapped=cv2.perspectiveTransform(p,H)
print(mapped)
# H maps src plane -> dst plane
# if reverse direction is needed:
Hinv=np.linalg.inv(H)
```

PERSPECTIVE CORNERS

```
h,w=template.shape[:2]
corners=np.float32(
    [[0,0],[w,0],[w,h],[0,h]]
).reshape(-1,1,2)
mapped=cv2.perspectiveTransform(corners,H)
# draw mapped polygon
```

13. SIFT + HOMOGRAPHY - FULL OBJECT/PANORAMA QUESTION

```
sift=cv2.SIFT_create()
kp1,d1=sift.detectAndCompute(g1,None)
kp2,d2=sift.detectAndCompute(g2,None)
bf=cv2.BFMatcher()
pairs=bf.knnMatch(d1,d2,k=2)
good=[m for m,n in pairs if m.distance<0.75*n.distance]
```

```
src=np.float32([kp1[m.queryIdx].pt for m in good]
).reshape(-1,1,2)
dst=np.float32([kp2[m.trainIdx].pt for m in good]
).reshape(-1,1,2)
H,inliers=cv2.findHomography(
src,dst,cv2.RANSAC,5.0)
print("good:", len(good),
"inliers:",int(inliers.sum()))
```

OBJECT CORNERS

```
h,w=img1.shape[:2]
corn=np.float32([[0,0],[w,0],[w,h],[0,h]]).reshape(-1,1,2)
mapped=cv2.perspectiveTransform(corn,H)
cv2.polylines(img2,[np.int32(mapped)],
True,(0,255,0),3)
# RANSAC removes geometrically inconsistent matches
```

14. TOUGHEST INTEGRATED QUESTION - MEDICAL IMAGE

Example:

```
# enhance -> denoise -> segment -> measure
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
enh=cv2.equalizeHist(gray)
den=cv2.GaussianBlur(enh,(5,5),0)
_,b=cv2.threshold(den,0,255,
cv2.THRESH_BINARY+cv2.THRESH_OTSU)
```

clean + contours

```
kernel=np.ones((3,3),np.uint8)
b=cv2.morphologyEx(b,cv2.MORPH_OPEN,kernel)
cs,_=cv2.findContours(
b,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)
for c in cs:
if cv2.contourArea(c)>500:
x,y,w,h=cv2.boundingRect(c)
print("area=",cv2.contourArea(c))
```

If illumination is uneven:

```
b=cv2.adaptiveThreshold(
enh,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY,11,2)
```

If objects touch:

```
# replace/simple threshold stage with
# watershed pipeline
```

15. TOUGHEST INTEGRATED QUESTION - LANE DETECTION

```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
blur=cv2.GaussianBlur(gray,(5,5),0)
edges=cv2.Canny(blur,50,150)
mask=np.zeros(gray.shape,np.uint8)
pts=np.array([[100,400],[300,250],
[500,250],[700,400]],np.int32)
cv2.fillPoly(mask,[pts],255)
roi=cv2.bitwise_and(edges,edges,mask=mask)
```

```
lines=cv2.HoughLinesP(
roi,1,np.pi/180,threshold=40,
minLineLength=40,maxLineGap=30)
for l in lines:
x1,y1,x2,y2=l[0]
cv2.line(img,(x1,y1),(x2,y2),(0,255,0),3)
# twist: change ROI points and
# Hough thresholds for the image
```

FULL LOGIC

```
image
-> grayscale
-> Gaussian blur
-> Canny
-> polygon ROI
-> HoughLinesP
-> draw lines
# If asked to "ignore sky/side regions":
# change polygon mask.
```

16. TOUGHEST INTEGRATED QUESTION - SECURITY + MOTION

previous + current frame

```
diff=cv2.absdiff(prev,gray)
_,motion=cv2.threshold(diff,30,255,
cv2.THRESH_BINARY)
motion=cv2.morphologyEx(
motion,cv2.MORPH_OPEN,
np.ones((3,3),np.uint8))
```

detect moving contours

```
cs,_=cv2.findContours(
motion,cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
for c in cs:
if cv2.contourArea(c)>500:
M=cv2.moments(c)
if M["m00"]:
p=(int(M["m10"]/M["m00"]),
int(M["m01"]/M["m00"]))
```

zone test

```
zone=np.array([[100,100],[500,100],
[500,400],[100,400]],np.int32)
if cv2.pointPolygonTest(zone,p,False)>=0:
print("ALERT: motion inside zone")
# motion + contour + centroid + polygon test
```

17. EXAM TRAPS - WHAT TO CHANGE

Question: uneven lighting
USE adaptiveThreshold
Question: automatic threshold
USE OTSU
Question: touching objects
USE watershed
Question: noisy edges
Gaussian -> Canny

Question: salt-and-pepper
medianBlur
Question: 3 point mapping
getAffineTransform + warpAffine
Question: 4 point perspective
findHomography + warpPerspective
Question: bad feature matches
ratio test + RANSAC

Question: texture
LBP
Question: shape/object gradients
HOG

Question: lines
HoughLinesP
Question: circles
HoughCircles
Question: multi-resolution
DWT

18. LAST-MINUTE HARD-CODE FORMULAS**AFFINE:**

$x' = A x + b$
3 point correspondences
warpAffine()

HOMOGRAPHY:

$x' \sim Hx$
4+ point correspondences
warpPerspective()
CONVOLUTION:
 $output = \text{floor}((N-K+2P)/S)+1$

CANNY:

Gaussian -> gradient -> NMS
-> double threshold -> hysteresis
SIFT:
scale-space -> keypoints
-> orientation -> descriptor
-> matching -> ratio test
-> RANSAC homography

SEGMENTATION:

threshold / edges / region growing
/ watershed / K-means / active contour
WAVELET:
DWT -> LL, LH, HL, HH
LL=low-frequency approximation
others=detail bands